

Phase 6: User Interface Development

Introduction

- ❖ In this phase, we focus on designing and building the **User Interface (UI)** for the *Skill Development and Employment Portal*. The goal is to provide an interactive, responsive, and user-friendly interface where **candidates, employers, and administrators** can seamlessly interact with the system.
 - ❖ This is achieved using Salesforce **Lightning Experience** features such as Lightning App Builder, Record Pages, Tabs, Utility Bar, and advanced customization with **Lightning Web Components (LWC)** integrated with **Apex**.
-

1. Lightning App Builder

The **Lightning App Builder** is used to create and customize application pages with drag-and-drop components.

- **Use Case in our Portal:**
- Create a **Candidate Dashboard Page** displaying job recommendations, applied jobs, and skill progress.
- Create an **Employer Dashboard Page** showing posted jobs, candidate applications, and recruitment analytics.
- **Implementation:**
- Custom pages are built with standard and custom components.
- Example: A "Skill Growth Tracker" component placed in the candidate's home page to show certifications and progress.

Goal

Creating app pages/dashboards for the three main personas (Candidate, Employer, Admin) using **Lightning App Builder**. Place a mix of standard components and placeholders for the LWCs you'll build later (CandidateProfileCard, JobRecommendationList, ApplicationTracker). Activate each page for the correct app/profile so users see the right UI.

Pre-reqs (from my earlier phases)

- Objects and fields exist from Phase 3: `Candidate__c`, `Job__c`, `Application__c`, `Skill__c`, `Certification__c` (if your API names differ, use those names).
- Any Apex classes you wrote earlier (e.g., `SkillManagerSafe`, `JobService`) are deployed — those will be wired to LWCs later.
- You have admin access to the org to create & activate pages.

What we'll do (step-by-step)

1. Open Lightning App Builder

- In Salesforce: click the **Gear (Setup)** icon → **Setup**.
- In Quick Find box type **Lightning App Builder** → open it.

Action	Label	Name	Namespace Prefix	Description	Type	Created By	Last Modified By
Edit Clone Del	Customer Account Page	Customer_Account_Page			Record Page	rups, 9/19/2025, 6:30 AM	rups, 9/19/2025, 6:39 AM
Edit Clone Del	Customer Account Pagee	Customer_Account_Pageee			Record Page	rups, 9/19/2025, 5:52 AM	rups, 9/19/2025, 6:10 AM
Edit Clone Del	dashborad	dashborad			Home Page	rups, 8/14/2025, 6:23 AM	rups, 8/14/2025, 6:24 AM
Edit Clone Del	Partner Account Page	Partner_Account_Page			Record Page	rups, 9/18/2025, 10:39 PM	rups, 9/19/2025, 6:46 AM
Edit Clone Del	Seller Home	Seller_Home			Home Page	rups, 8/14/2025, 6:07 AM	rups, 8/14/2025, 6:07 AM
Edit Clone Del	Travel Approval Record Page	Travel_Approval_Record_Page			Record Page	rups, 9/18/2025, 4:14 AM	rups, 9/18/2025, 4:14 AM
Edit Clone Del	Travel Approval Record Page	Travel_Approval_Record_Page2			Record Page	rups, 9/18/2025, 5:03 AM	rups, 9/18/2025, 5:03 AM
Edit Clone Del	Travel Approval Record Page	Travel_Approval_Record_Page1			Record Page	rups, 9/18/2025, 4:57 AM	rups, 9/18/2025, 4:57 AM

2. Create a new App Page for Candidate Dashboard

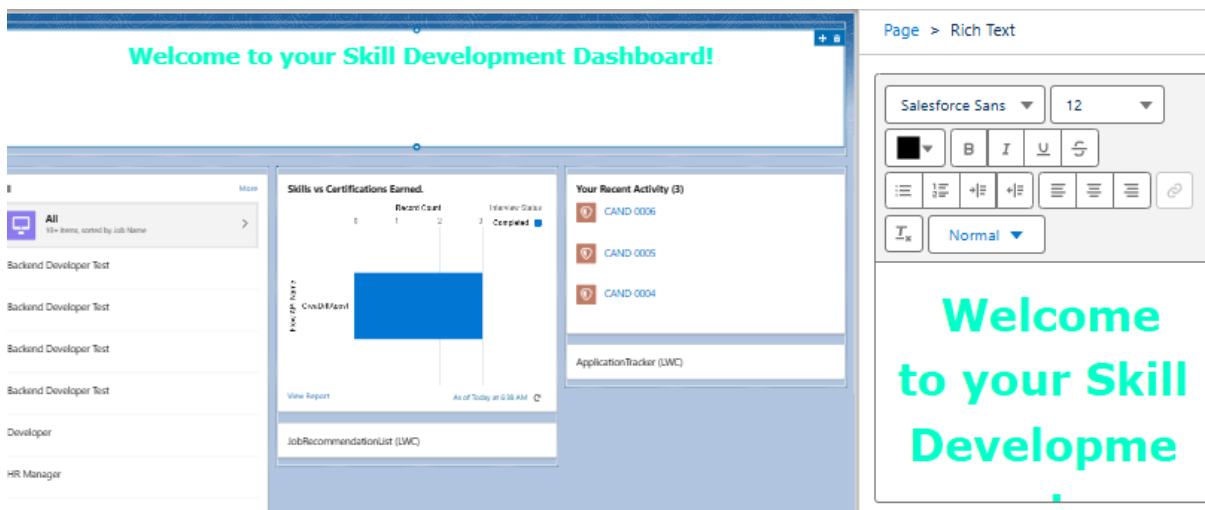
- Click **New** → choose **App Page** → **Next**.
- Choose a layout (recommend: **Header + Three Columns** or **One Region with Sidebar**).
- Rationale: Candidate dashboard benefits from a header (name, quick actions) + 2–3 columns (profile, recommendations, activity).

- Name it: `Candidate Dashboard - SkillDevPortal`.
- Click **Finish** to open the builder canvas.

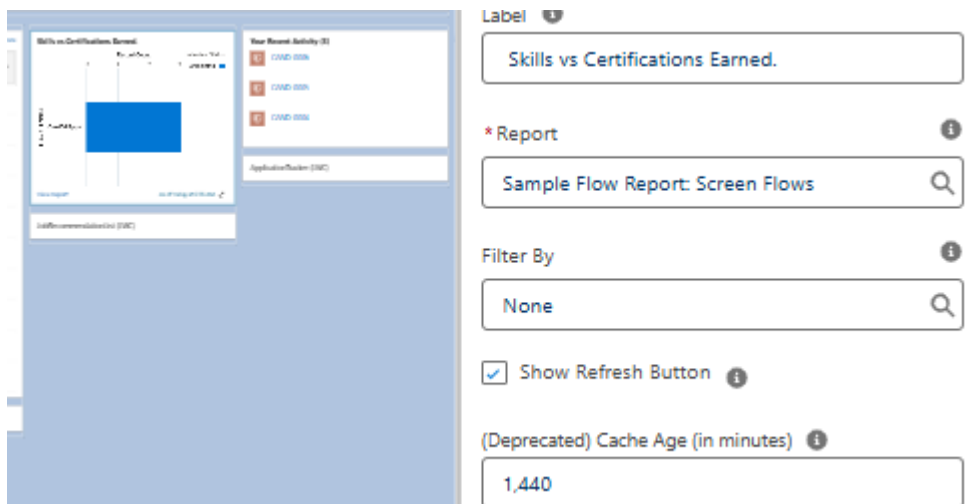


3. Add components onto the canvas

- Left panel: drag standard components first:
- **Rich Text** (header area) — add a welcome message and use tokens like `Hello, {{$User.FirstName}}` (or leave placeholder).



- **Report Chart** or **List View** — show "Recommended Jobs" or "Open Jobs".
- **Related Lists - Single** (if candidate record context will be used later).



- Add **Custom Component** placeholders: drag a **Custom** component area and label it `CandidateProfileCard (LWC)` and `JobRecommendationList (LWC)` — these will be replaced by your actual LWCs later.

Label ⓘ

Your Recent Activity

* Objects

Application Candidate Job

Select...

Number of Records to Display

3

Welcome to your Skill Development Dashboard!

All More

All
5+ Items, sorted by Job Name

Backend Developer Test

Backend Developer Test

Backend Developer Test

Backend Developer Test

Developer

CandidateProfileCard (LWC)

Skills vs Certifications Earned.

Record Count

Interview Status

Completed

Flow API Name

Created/Appvi

View Report

As of Today at 6:38 AM

JobRecommendationList (LWC)

Your Recent Activity (7)

CAND-0006

CAND-0005

CAND-0004

CAND-0001

CAND-0002

Backend Developer Test

Backend Developer Test

ApplicationTracker (LWC)

4. Configure component properties

- For each component, set properties to match your data model:
- Example: For a `List View` component, set `Object = Job__c`, `Filter = Active = true`.
- For custom LWC placeholders, add a **Component Name** and a short `Label` in properties.
- If a component accepts a `recordId` or `userId`, set the property to use `$User.Id` (or plan to pass it from record pages).

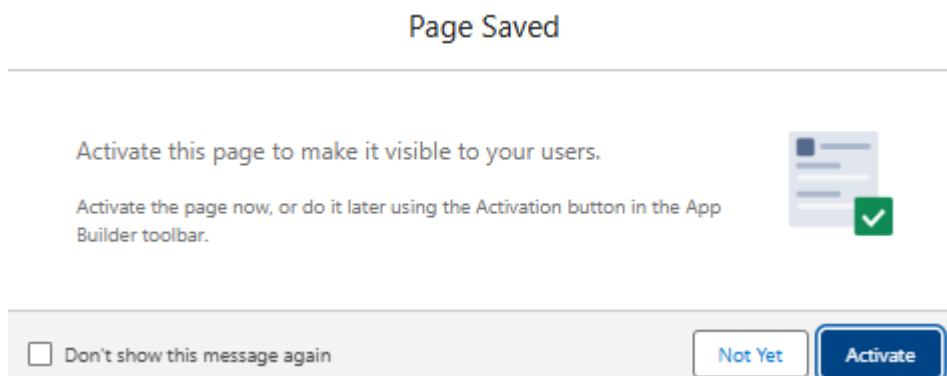
Filters

1 User > Active Equal true

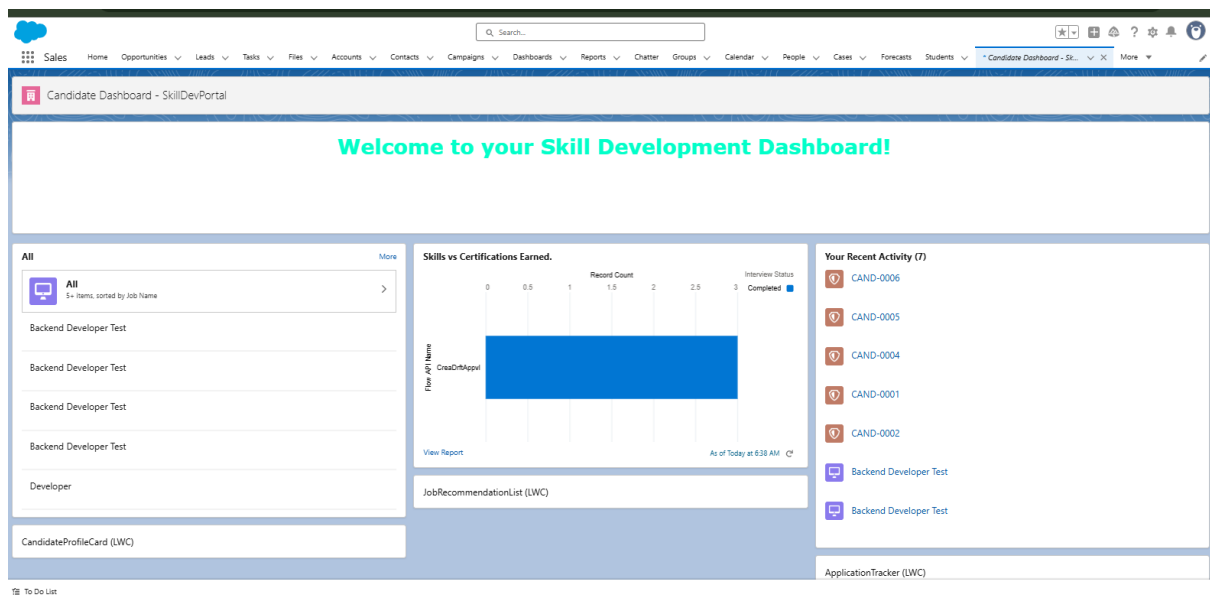
+ Add Filter

5. Save and Activate

- Click **Save**.
- Click **Activation** → Choose how this page is used:



- **App Default:** select the Lightning App (e.g., *SkillDevPortal App*) and set **App Default** if you want it as the app's default page.
 - **App and Profile:** assign the Candidate Dashboard only to the *Candidate* profile (or a permission set) so only candidates see it.
 - You can also set it as **Org Default** or assign to specific record pages — for now use **App & Profile** to scope to Candidate users.
 - Click **Activate** → confirm assignments → **Done**.
 - Screenshot to take: Activation dialog showing App/Profile assignment.
6. **Verify in Lightning Experience**
- Open **App Launcher**, switch to the SkillDevPortal app (or the app you assigned), and open the Candidate Dashboard.
 - Confirm components render (placeholders will show until LWCs built).



2. Record Pages

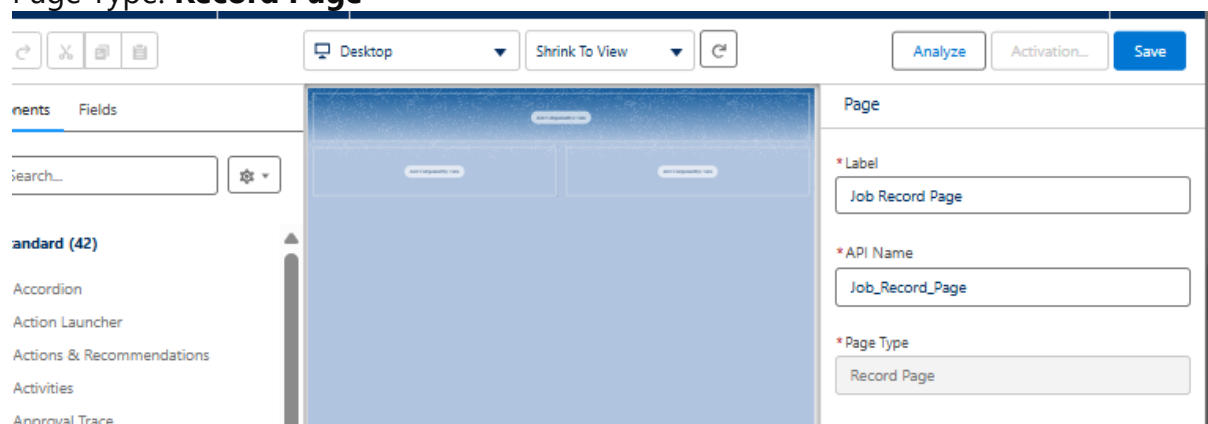
- Record pages define the layout of data records.
- Now that the **App Page (3-column console)** is set, the next step is to configure **Record Pages** for your key objects.

1. Go to Record Pages

1. In Salesforce Setup → search **App Builder**.
2. Select **Lightning App Builder**.
3. At the top, switch from **App Page** to **Record Page**.
4. Click **New Record Page**.

2. Create a Record Page for Job__c

- Name: **Job Record Page**
- Object: **Job__c**
- Page Type: **Record Page**



Layout to build:

- **Left Panel:** Highlights Panel (shows job title, status, key fields)

Page > Field Section

Label ⓘ

Job Position Details

Columns

☐ 1 Column

☒ 2 Columns

☐ Align fields horizontally ⓘ

▼ Set Component Visibility

Filters

+ Add Filter

- **Middle Panel:** Record Details (standard job fields)

Page > Field Section

Label ⓘ

Job Position Details

Columns

☐ 1 Column

☒ 2 Columns

☐ Align fields horizontally ⓘ

▼ Set Component Visibility

Filters

+ Add Filter

- **Tabs:** Add tabs for
- **Candidates (Related List)**
- **Skills Required (custom related list if you added)**
- **Right Panel:** Activity (Log a Call, Email, Notes, etc.)

3. Create a Record Page for Candidate__c

- Name: **Candidate Record Page**
- Object: **Candidate__c**
- Page Type: **Record Page**
- Assign to: **Recruitment App**

Layout to build:

- **Left Panel:** Highlights Panel (Candidate name, email, phone)
 - **Middle Panel:** Record Detail (candidate info, experience, resume link)
 - **Tabs:** Add tabs for
 - **Skills (related list)**
 - **Certifications (related list)**
 - **Applications (junction object Job–Candidate)**
 - **Right Panel:** Activity timeline
-

4. (Optional) Create a Record Page for Application__c

If you made a junction object (Job Application):

- Shows **Job__c lookup**, **Candidate__c lookup**, **Status**, **Interview Date** etc.

▼ Set Component Visibility

Filters

1

Record > Application Status
Contains Submitted

×

2

Record > Application Status
Contains Shortlisted

×

3

Record > Application Status
Contains Pending

×

4

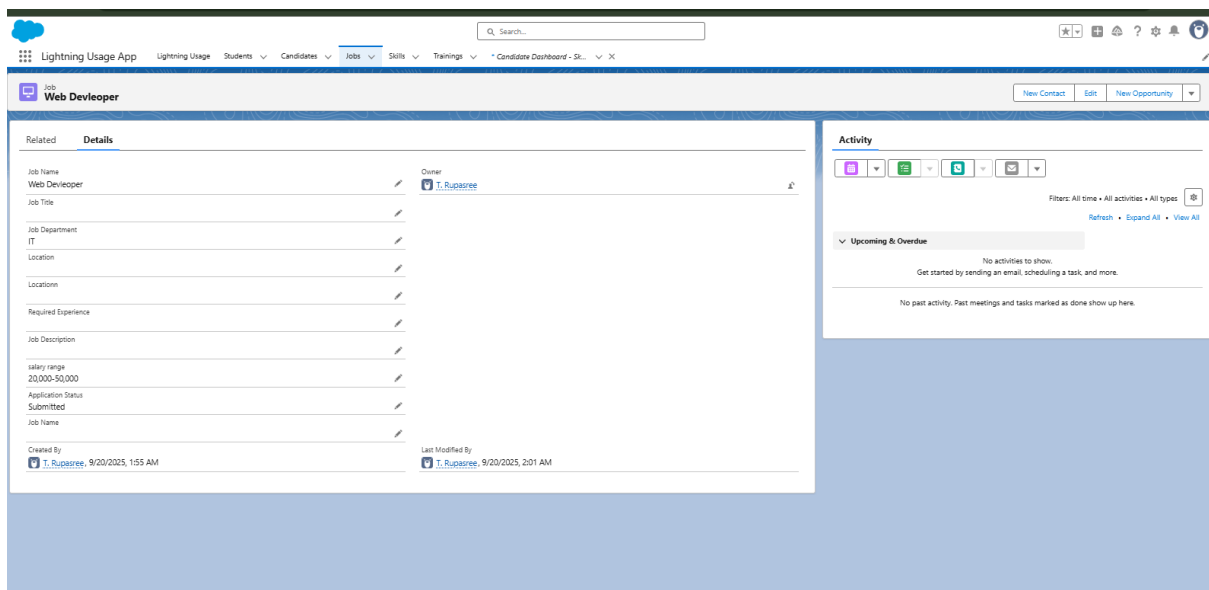
Record > Application Status
Contains Rejected

×

+ Add Filter

- **Use Case in Portal:**
- Candidate record page → Displays personal details, skills, certifications, and job applications.
- Job record page → Shows job description, required skills, employer details, and candidate applications.
- **Implementation:**

- Customize record detail pages using related lists (e.g., Candidate → Job Applications, Employer → Posted Jobs).
- Add quick actions like "Apply for Job" or "Schedule Interview."



3. Tabs

- Tabs provide structured navigation within the portal.
- **Tabs** are UI elements that let users navigate between different views or data objects within an app. There are three main types:

1. **Standard Object Tabs** – for Salesforce standard objects like Accounts, Contacts, Leads.
2. **Custom Object Tabs** – for your custom objects like Candidate__c, Job__c, Skill__c, Certification__c.
3. **Visualforce / Lightning Page Tabs** – for custom pages, dashboards, or LWCs.

In your portal, you will use:

- **Candidate Tabs:** Dashboard, My Profile, Skills, Job Applications, Certifications.
- **Employer Tabs:** Dashboard, Post Job, Manage Applications, Candidate Search.
- **Admin Tabs:** Reports, Skill Database, Approvals, Analytics.

◆ Step 2 — Creating Custom Object Tabs

1. Go to Setup → Tabs

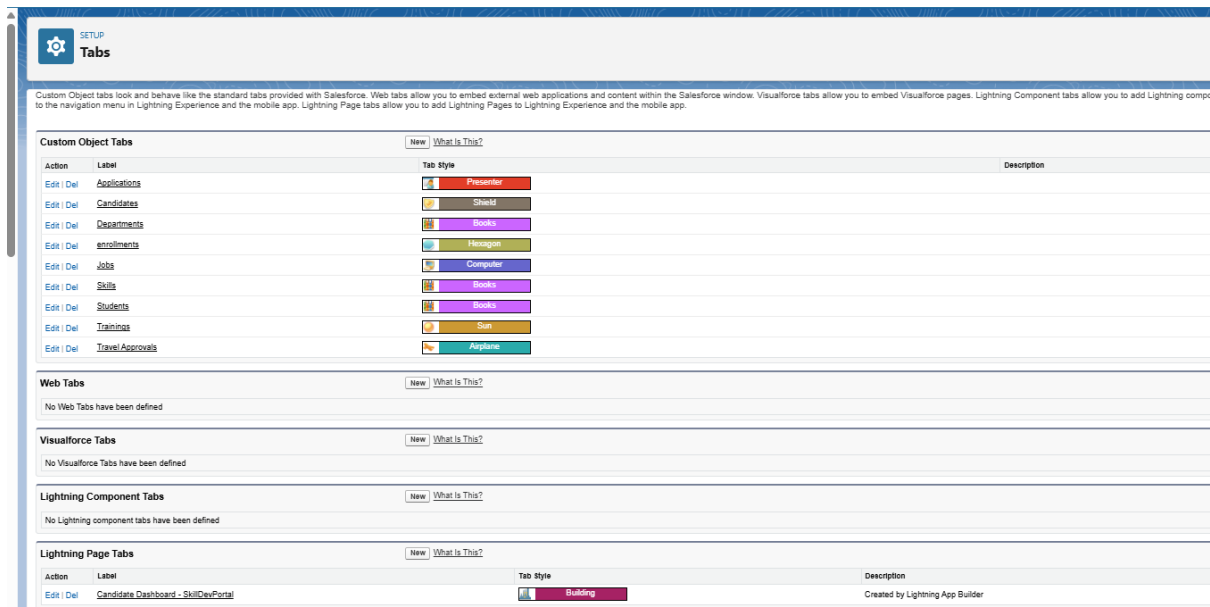
- Search: **Tabs** in Quick Find.
- You'll see sections: Custom Object Tabs, Web Tabs, Visualforce Tabs.

2. Create Custom Tabs for Objects

- Click **New** under **Custom Object Tabs**.
- **Object:** Choose your custom object (e.g., Candidate__c).
- **Tab Style:** Pick an icon/color.
- **Tab Name:** Candidate.
- Click **Next** → **Save**.

Repeat for:

- Job__c → Tab Name: Job
- Skill__c → Tab Name: Skill
- Certification__c → Tab Name: Certification
- Application__c → Tab Name: Applications



◆ Step 3 — Adding Tabs to Lightning App

1. Go to **App Manager** → Find your app (e.g., SkillDevPortal) → **Edit**.
2. Navigate to **Navigation Items**.
3. **Add the Tabs** you created:

Add Items

Available Items

Favorites

All2

Search all items...

Applications Xenrollments X

2 items selected

+Accounts

+Activation Targets

+Activations

+All Sites

+Alternative Payment Methods

+Analytics

+App Launcher

+Applications

+Appointment Categories

+Appointment Invitations

+Approval Requests

+Approval Submission Details

+Approval Submissions

+Approval Work Items

+Asset Action Sources

Cancel

Add 2 Nav Items

- Candidate, Job, Skill, Certification, Application.
4. Reorder tabs for each persona:
 - Candidate: Dashboard → My Profile → Skills → Certifications → Applications.
 - Employer: Dashboard → Post Job → Manage Applications → Candidate Search → Reports.
 5. Click **Save & Finish**.

Edit Lightning Usage App App Navigation Items

Personalize your nav bar for this app. Reorder items, and rename or remove items you've added.

[Learn More](#) ⓘ

ⓘ 2 items added to your list. Save your updates.

NAVIGATION ITEMS (8)

[Add More Items](#)

≡		Lightning Usage	
≡		Students	
≡		Candidates	
≡		Jobs	
≡		Skills	
≡		Trainings	
≡		Applications	×
≡		enrollments	×

[Reset Navigation to Default](#) ⓘ

Cancel

Save

◆ Step 4 — Tabs on Record Pages or Dashboards (Lightning App Builder)

Sometimes within a **Record Page** or **Dashboard App Page**, you want to organize components in **tabs**.

Example: Candidate Record Page Tabs

- Drag **Tabs** component from the **Standard Components** panel.
- Inside Tabs:
 1. **Tab 1 → Candidate Details**
 - Drag **Record Detail** component here.
 - Shows personal info, email, phone, resume.

Page > Tabs

Label ?

Candidate Details

Default Tab

Dashboard

Tabs

Chatter

Dashboard

Add Tab

2. Tab 2 → Skills

- Drag **Related List – Single**
- Configure: Related List = Skill__c (Candidate Skills)

3. Tab 3 → Certifications

- Drag **Related List – Single**
- Configure: Related List = Certification__c

4. Tab 4 → Applications

- Drag **Related List – Single**
- Configure: Related List = Application__c

Trainings		New Import Assign Label	
Recently Viewed ▼ ↑		Search this list...	
2 Items • Updated a few seconds ago		⚙️ 📄 🔄 ✎ 🗑 🔍	
☐ Training Name			
1	☐ Verbal Communications		
2	☐ Java Full Stack		

Example: Job Record Page Tabs

- Tab 1 → Job Details (Record Detail)
- Tab 2 → Candidate Applications (Related List)
- Tab 3 → Required Skills (Related List)

✂ Tip: You can rename the **Tab Labels** in the properties panel (e.g., “My Skills”, “Applications History”) to make it more user-friendly.

◆ Step 5 — Tabs in LWCs

If you're building **Lightning Web Components** for dashboards:

- Use `lightning-tabset` and `lightning-tab` in your LWC HTML.
 - Example: Candidate Dashboard LWC with Tabs:

```
<lightning-tabset> <lightning-tab label="Recommended Jobs"> <c-job-recommendation-list></c-job-recommendation-list> </lightning-tab>
<lightning-tab label="My Skills"> <c-candidate-skill-list candidate-id={recordId}></c-candidate-skill-list> </lightning-tab> <lightning-tab
label="Applications"> <c-application-tracker candidate-id={recordId}></c-application-tracker> </lightning-tab> </lightning-tabset>
```
 - Each tab loads a separate LWC component dynamically.
 - Can include **@wire or imperative Apex** calls in each component to fetch data. We will see them in our further steps
-

◆ Step 6 — Key Benefits for Your Portal

1. **Organized UI** – Users don't need to scroll endlessly.
 2. **Persona-specific Navigation** – Candidate, Employer, Admin tabs show relevant info only.
 3. **Dynamic Data Display** – Tabs can include Record Detail, Related Lists, and LWCs.
 4. **Easy Expansion** – Add more tabs later (e.g., "Interview Feedback", "Job Alerts").
-
- **Use Case in Portal:**
 - **Candidate Tabs:** Dashboard, My Profile, Skills, Job Applications, Certifications.
 - **Employer Tabs:** Dashboard, Post Job, Manage Applications, Candidate Search.
 - **Admin Tabs:** Reports, Skill Database, Approvals, Analytics.
 - **Implementation:**
 - Created custom tabs for objects: Candidate, Job, Skill, Certification, and Application.
 - Tabs linked to record pages for seamless navigation.
-

4. Home Page Layouts

- The Home Page is customized to act as the **central hub** for different users.
- The **Home Page** is the first thing users see when they log in to the portal. Customizing it makes your portal **personalized, intuitive, and actionable**. You can create **different home pages for Candidates, Employers, and Admins**.

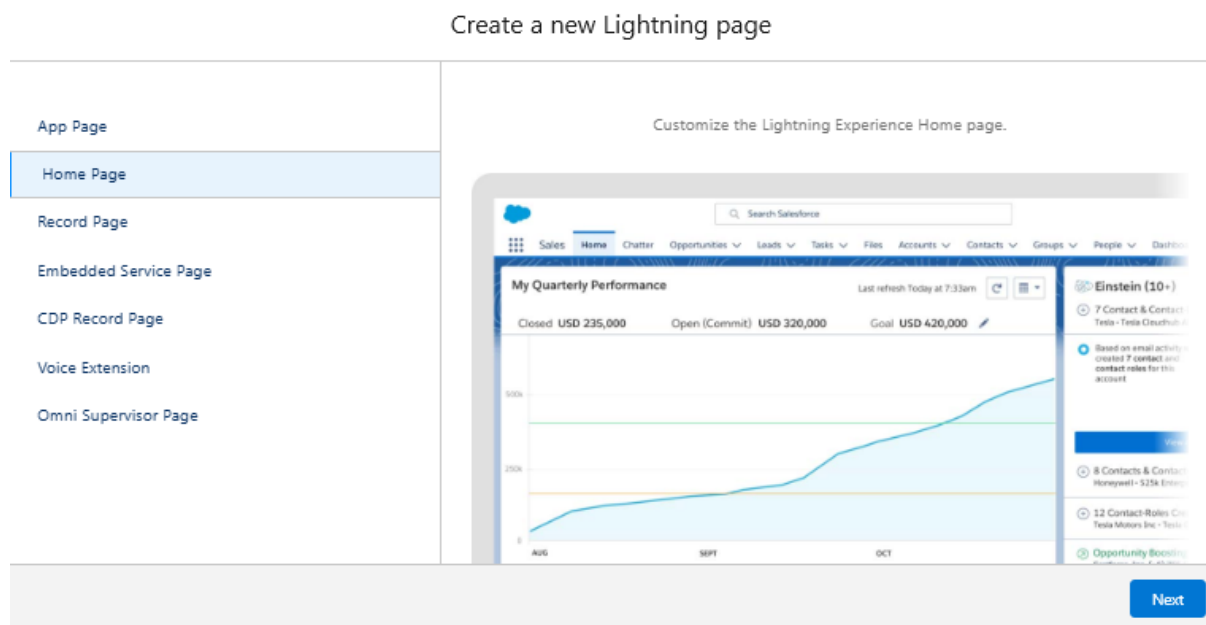
Why Customize Home Pages

- Provide a **central hub** for users.
- Highlight **key metrics, tasks, and reports**.
- Add **quick actions** to improve workflow efficiency.
- Incorporate **custom LWCs** to show dynamic content (jobs, skill progress, application tracker).

Step-by-Step: Create a Home Page Layout

1□ Open Lightning App Builder for Home Page

- Click **Gear Icon** → **Setup**.
- Search **Lightning App Builder**.
- Click **New** → **Home Page**.



2□ Choose a Layout

Salesforce provides standard layouts:

- **Header + Three Columns** → Dashboard style, good for candidates.
- **Header + Two Columns** → Balanced layout for employers.
- **One Column** → Simple layout, good for Admin home page.

Recommendation:

- Candidate → Header + Three Columns
- Employer → Header + Two Columns
- Admin → One Column (focus on reports & analytics)

☐ Drag-and-Drop Components Into Layout

Candidate Home Page Layout

Header Region

- Drag **Rich Text** or your **candidateWelcome LWC**.
- Purpose: Display greeting and motivational text.

Left Column

- Drag **List View** → Object = Job__c, List View = "Active Jobs"
- Shows all currently open jobs.

Middle Column

- Drag **Report Chart** → e.g., "Skills vs Certifications Earned"
- Shows skill progression graphically.

Right Column

- Drag **Recent Items** → Object = Application__c
- Shows recently applied jobs.
- Drag **Custom LWC Placeholder** → ApplicationTracker (to monitor application status).

Employer Home Page Layout

Header

- Rich Text or LWC → "Welcome, [Employer Name]! Here's your dashboard."

Left Column

- **Report Chart** → e.g., "Open Jobs vs Applications"
- Displays analytics on posted jobs.

Right Column

- **List View** → Job__c object, filter: Jobs posted by this employer.

- Custom LWC placeholder → CandidateShortlist (to quickly view recommended candidates).
-

Admin Home Page Layout

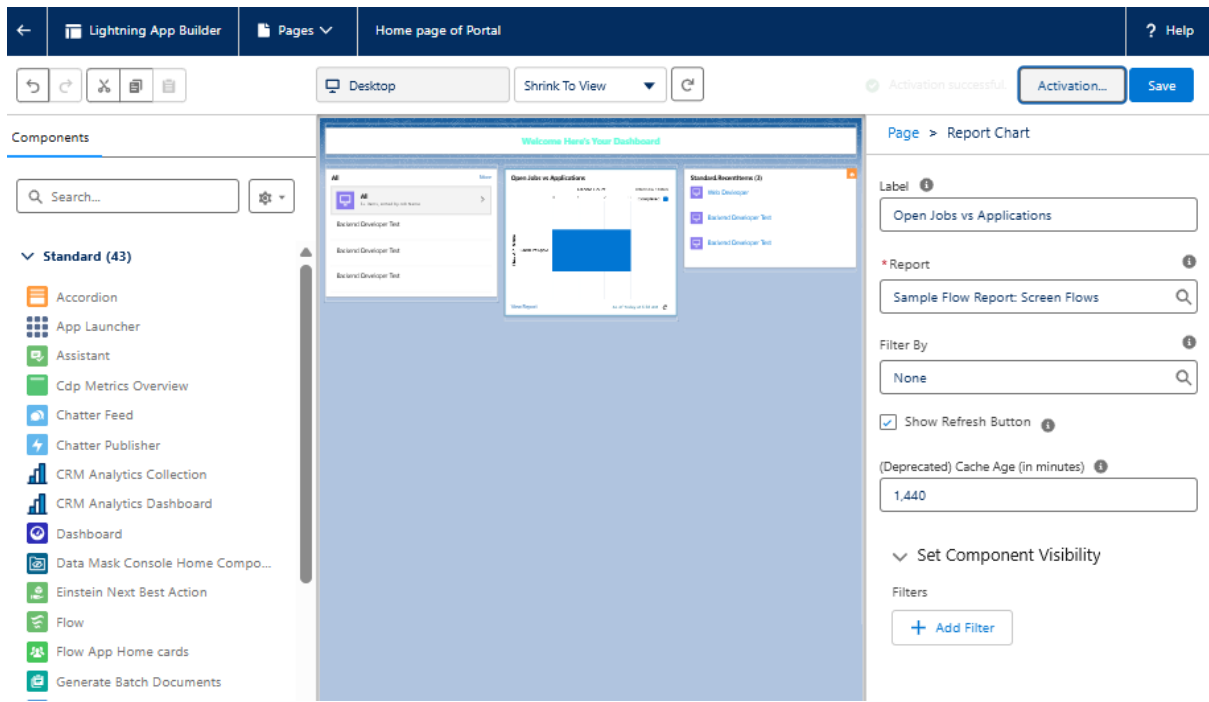
- Layout can be simple → One column, focusing on reports.
 - Drag **Report Charts**:
 - Total Candidates
 - Total Jobs Posted
 - Skills Added This Month
 - Quick Actions:
 - "Add Skill"
 - "Approve Certification"
 - Custom LWC placeholder → AdminAnalyticsDashboard (combines key metrics).
-

4 Configure Component Properties

- For **List Views** → select the object and the view filter.
 - For **Report Charts** → select the report you created in Phase 3.
 - For **Custom LWCs** → make sure `<isExposed>true</isExposed>` in meta.xml.
 - Set **recordId** or **userId** in LWC if dynamic content is needed.
-

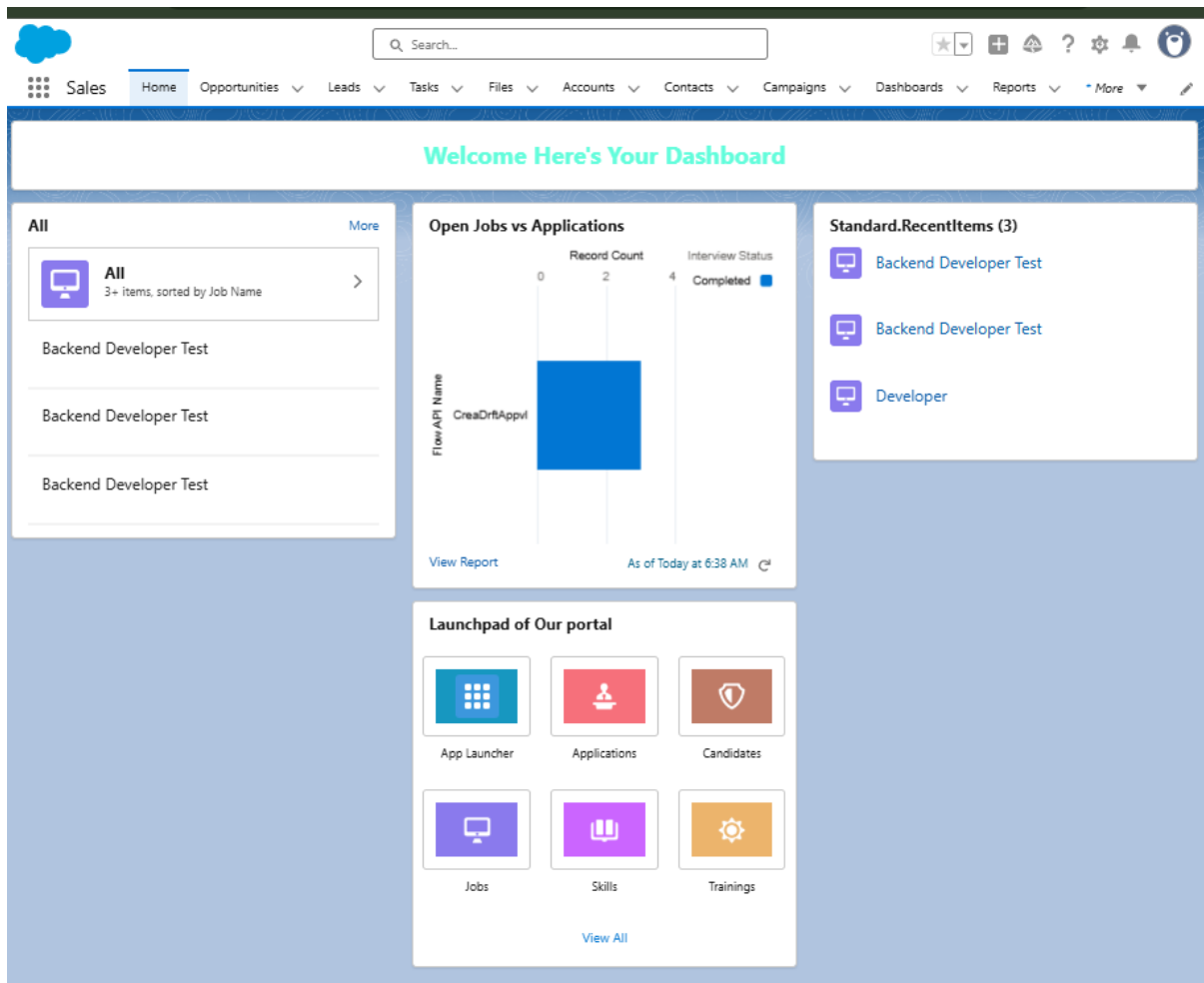
5 Save and Activate Home Page

1. Click **Save**.
2. Click **Activation** → Assign to:
 - **App Default** (all users of the app)
 - **Profile** (different pages for Candidate, Employer, Admin)
3. Confirm → Done.



6 Verify in Lightning Experience

- Open **App Launcher** → **SkillDevPortal App**.
- Check each user profile:
- Candidate → sees personalized dashboard
- Employer → sees job analytics
- Admin → sees reports and approval actions



- **Use Case in Portal:**
- Candidate Home Page → Personalized job recommendations, upcoming interviews, latest skill courses.
- Employer Home Page → Pending applications, recommended candidates, analytics on job postings.
- **Implementation:**
- Used standard components like "Recent Items," "Reports," and custom components like "Recommended Jobs."
- Personalized layouts depending on user profiles (Candidate / Employer / Admin).

5. Utility Bar

- The Utility Bar provides quick access to commonly used tools.
- The **Utility Bar** is a fixed, bottom-of-the-screen bar in Lightning Apps that provides quick access to tools and components. In your portal, this is perfect

for actions like **Quick Job Search**, **Update Skills**, **Help/FAQ**, or even a **Chatbot**.

1 Where to Find the Utility Bar

1. Go to **Setup** → **App Manager**
 2. Find your **SkillDevPortal App**
 3. Click **Edit** (lightning app icon → Edit)
 4. On the left panel, click **Utility Bar**
 - This is where you add components to the bar that appears at the bottom of the Lightning app
-

2 Add Components to the Utility Bar

Common Utility Bar Items for SkillDevPortal:

Component	Purpose	Notes
Global Search (Standard)	Search Jobs, Candidates	Use standard component
Custom LWC: Quick Add Skill	Add skill without leaving page	Create LWC and expose it
Custom LWC: Skill Bot / Chatbot	Help candidates & employers	LWC with messaging interface
Recent Items	Quickly access recent applications or jobs	Standard component
Rich Text / Info	Display tips, shortcuts	Optional

Step-by-Step: Adding a Utility Bar Component

1. Click **Add** in Utility Bar
2. Choose Component type:
 - **Standard** → Recent Items, Report Chart, or Utility items
 - **Custom** → Any LWC you deployed (e.g., `QuickAddSkill`, `SkillBot`)
3. Configure Properties:
 - **Label** → Quick Add Skill
 - **Icon** → Choose from Salesforce icon library
 - **Height / Width** → If the component is a popup panel
 - **Default Panel Open** → Optional (usually false)
4. Repeat for all items you want in the bar
5. **Drag to reorder** — the leftmost icon is first in the bar

The screenshot displays the 'App Settings' interface for 'SkillDevPortal'. The 'Utility Items (Desktop Only)' section is active, showing a list of utility items and a properties panel for the selected 'To Do List' item.

Utility Bar Alignment: Default

Utility Items List:

- To Do List
- Quip Notifications
- Recent Items
- History
- Chatter Feed
- Report Chart

Properties Panel for 'To Do List':

- Label:** To Do List
- Icon:** task
- Panel Width:** 427
- Panel Height:** 680
- ☒ Start automatically

3 Use Case Examples for our Portal

1. **Candidate Perspective**
 - Quick access to update skills
 - Chatbot for career guidance
 - Search jobs without leaving current page
 2. **Employer Perspective**
 - Search candidates quickly
 - Open analytics reports
 - Access “Pending Approvals” for applications
 3. **Admin Perspective**
 - Open Skill Database
 - Review pending approvals
 - Access system-wide reports
-

4 Activate the Utility Bar

1. Once all items are added, **Save**
2. Click **Back** → your App → **Lightning Experience**
3. At the **bottom of the screen**, you should now see the **Utility Bar** icons



Search...



SkillDevPortal

Accounts

App Launcher

Applications

Candidate Dashboard - SkillDevPortal

Candidates

More

Welcome to your Skill Development Dashboard!

All

More



All

5+ Items, sorted by Job Name



Backend Developer Test

Backend Developer Test

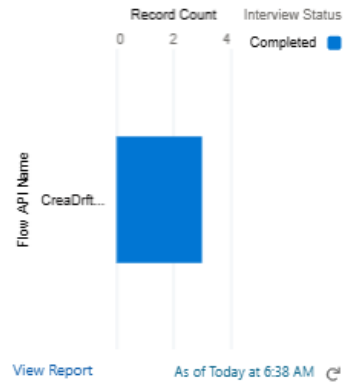
Backend Developer Test

Backend Developer Test

Developer

CandidateProfileCard (LWC)

Skills vs Certifications Earned.



JobRecommendationList (LWC)

Your Recent Activity (7)



Backend Developer Test



Backend Developer Test



Developer



Web Developer



Backend Developer Test



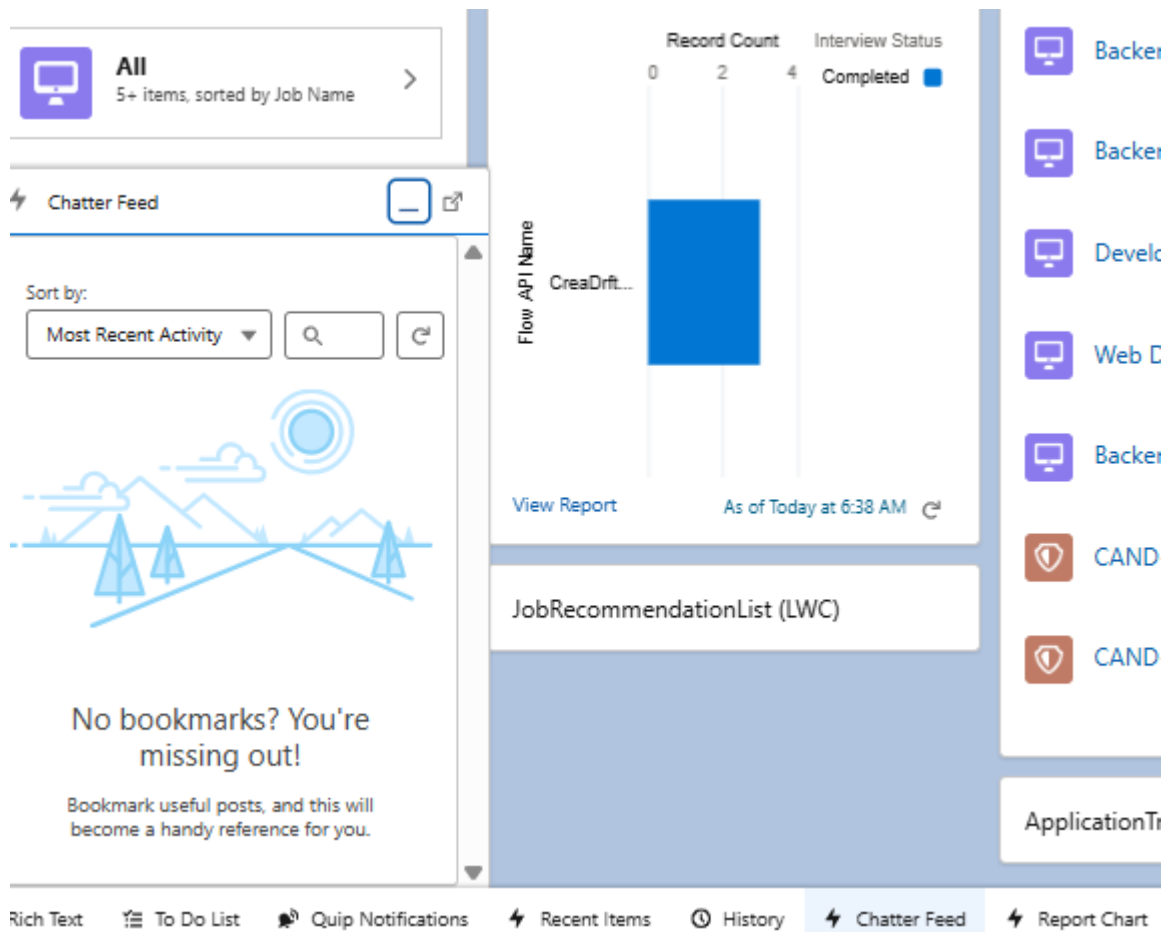
CAND-0006



CAND-0005

ApplicationTracker (LWC)

Rich Text To Do List Quip Notifications Recent Items History Chatter Feed Report Chart



- **Use Case in Portal:**
- Chatbot for support (Skill Bot).
- Quick "Job Search" option.
- Shortcut to "Update Skills."
- **Implementation:**
- Configured a **Utility Bar** with custom LWC components for:
- Global Search across Jobs and Candidates.
- Quick Add Skill.
- Help & FAQ section.

6. Lightning Web Components (LWC)

LWCs provide modern, reusable UI components.

Step 1. What is LWC?

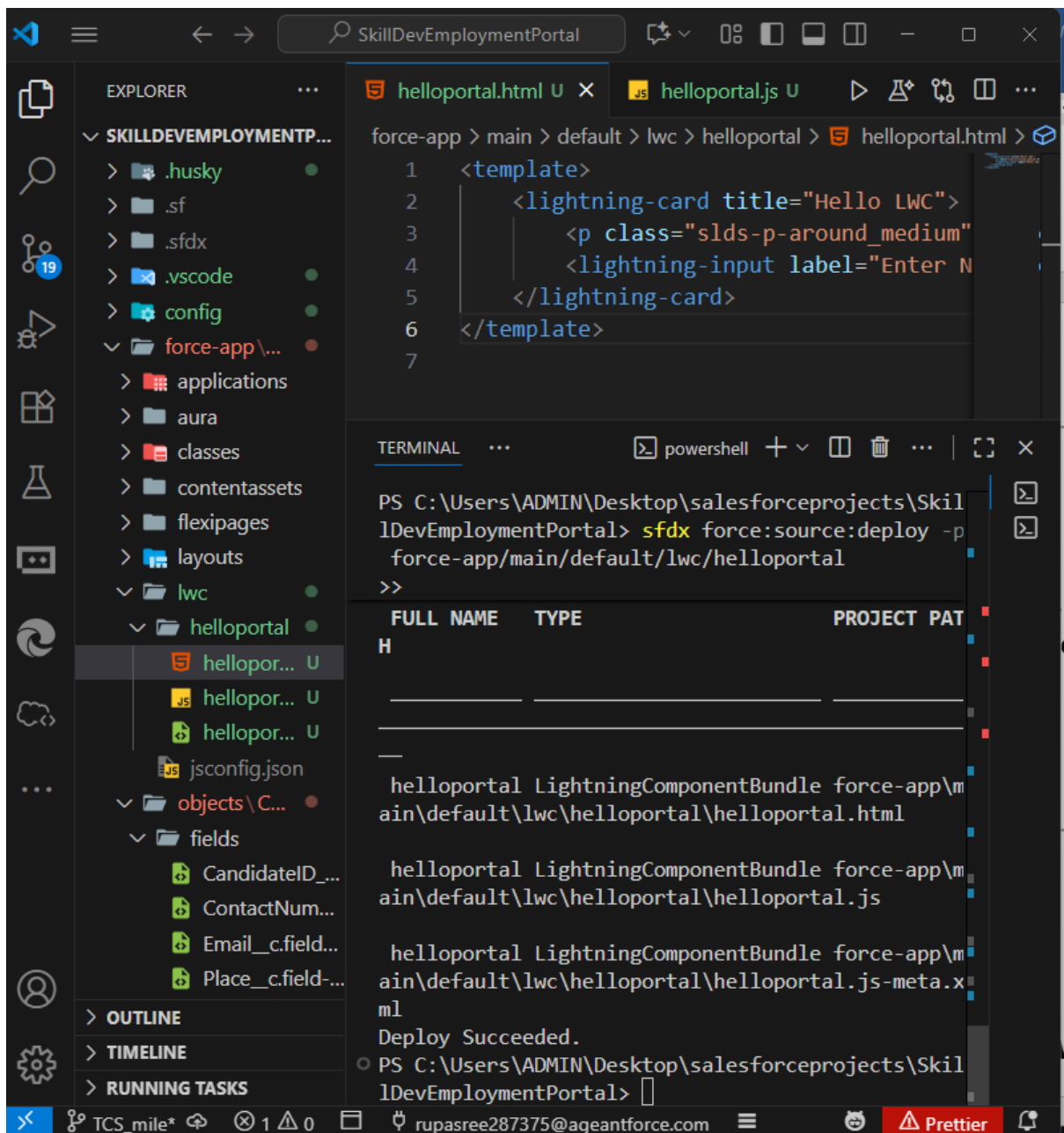
- LWC is Salesforce's modern framework for building **UI components**.
- It's built on **web standards (HTML, CSS, JavaScript)**.
- Can be placed on **Record Pages, Home Pages, App Pages, Utility Bar**, etc.

Step 2. Prerequisites

1. Install **VS Code** with **Salesforce Extensions**.
 2. Connect your Salesforce org with **SFDX (Salesforce CLI)**.
 3. Open your project folder in VS Code.
-

Step 3. Create a New LWC

1. In VS Code:
 - Right click `lwc` folder → **Create Lightning Web Component**.
 - Give it a name, e.g., `helloportal`.
2. It creates 3 files:
 - `helloportal.html` → UI template
 - `helloportal.js` → JS logic
 - `helloportal.js-meta.xml` → Visibility metadata



Step 4. Basic Example

helloWorld.html

```

<template> <lightning-card title="Hello LWC"> <p class="slds-p-around_medium">Hello, {name}!</p> <lightning-input label="Enter Name"
onchange={handleChange}></lightning-input> </lightning-card> </template>

```

helloWorld.js

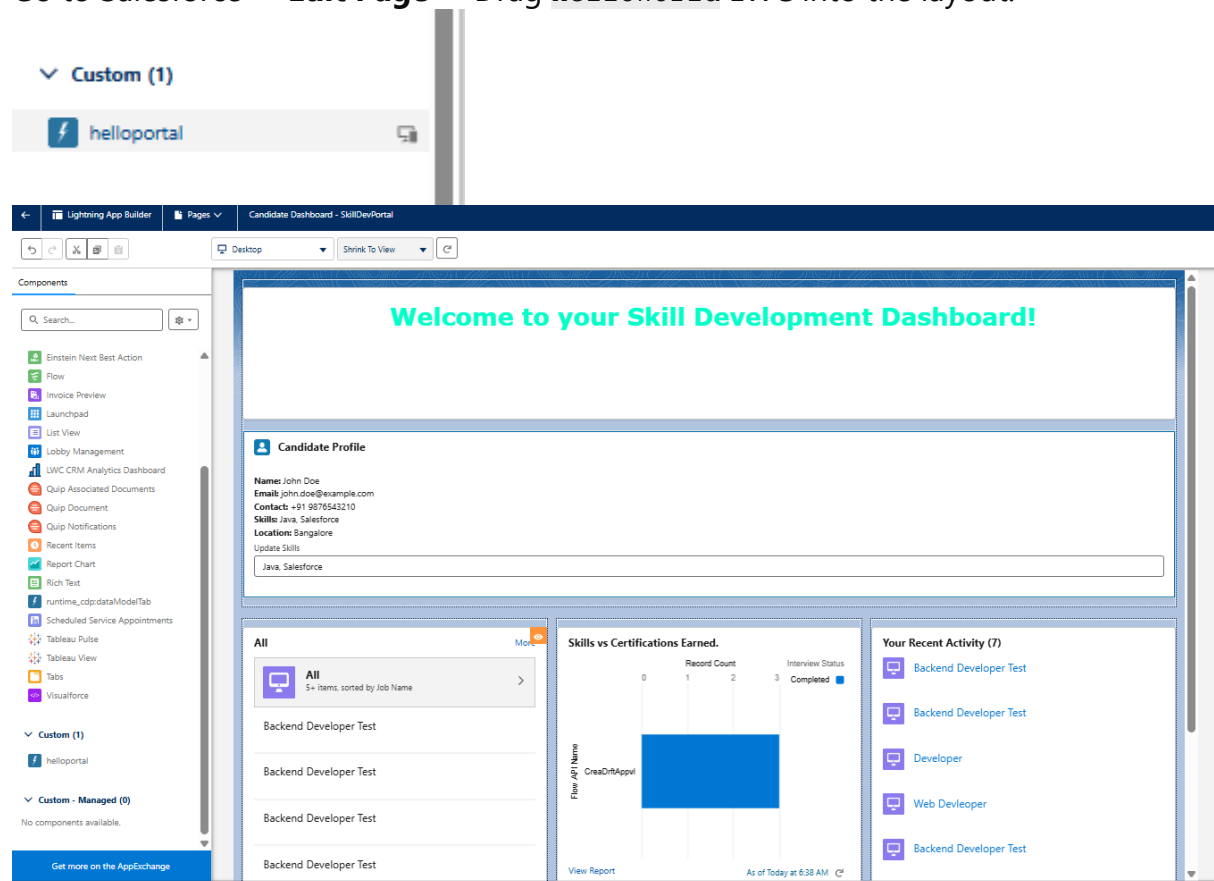
```
import { LightningElement, track } from 'lwc'; export default class
HelloWorld extends LightningElement { @track name = 'World';
handleChange(event) { this.name = event.target.value; } }
```

helloWorld.js-meta.xml

```
<?xml version="1.0" encoding="UTF-8"?> <LightningComponentBundle
xmlns="http://soap.sforce.com/2006/04/metadata">
<apiVersion>60.0</apiVersion> <isExposed>true</isExposed> <targets>
<target>lightning__RecordPage</target> <target>lightning__AppPage</target>
<target>lightning__HomePage</target> </targets> </LightningComponentBundle>
```

Step 5. Deploy to Salesforce

- Right-click component → **SFDX: Deploy Source to Org.**
- Go to Salesforce → **Edit Page** → Drag helloWorld LWC into the layout.



- Save & Activate.

Step 6. Try It

- Open the page → You'll see a card:
- Type a name → It updates dynamically.

The screenshot displays the SkillDevPortal interface. At the top, there is a navigation bar with the SkillDevPortal logo and several menu items: Accounts, Candidate Dashboard - SkillDevPortal, App Launcher, Applications, and Candidates. A large green 'Welcome' message is visible on the right side of the header. Below the header, there is a section titled 'All' with a 'More' link. This section contains a list of job cards, each with a computer icon and the text 'Backend Developer Test'. Below this list, there is a 'CandidateProfileCard (LWC)' section. This section contains a 'Candidate Profile' card, which is highlighted with a red rounded rectangle. The candidate profile card displays the following information: Name: John Doe, Email: john.doe@example.com, Contact: +91 9876543210, Skills: Java, Salesforce, and Location: Bangalore. Below the profile information, there is an 'Update Skills' button and a text input field containing 'Java, Salesforce'.

SkillDevPortal Accounts Candidate Dashboard - SkillDevPortal App Launcher Applications Candidates

Welcome

All More

Backend Developer Test

Backend Developer Test

Backend Developer Test

Backend Developer Test

Developer

CandidateProfileCard (LWC)

Candidate Profile

Name: John Doe
Email: john.doe@example.com
Contact: +91 9876543210
Skills: Java, Salesforce
Location: Bangalore

Update Skills

Java, Salesforce

What We Achieved

- Created a **basic LWC**.
 - Learned about **HTML, JS, XML structure**.
 - Exposed it to pages.
 - Saw how **data binding** works in LWC.
 - **Use Case in Portal:**
 - **CandidateProfileCard**: Displays candidate details and skills.
 - **JobRecommendationList**: Shows recommended jobs based on skills.
 - **ApplicationTracker**: Monitors the status of job applications.
 - **Implementation:**
 - Designed LWCs with HTML, JS, and Apex integration for dynamic data display.
-

7. Apex with LWC

Apex is used when LWC needs server-side operations.

- **Use Case in Portal:**
- Fetching all jobs matching candidate skills.
- Returning shortlisted candidates for an employer's job posting.
- **Implementation Example:**
- `getRecommendedJobs(candidateId)` Apex method called from LWC.
- Data retrieved and rendered dynamically in the UI.
-

1 Apex Class (Server-Side)

Suppose you want to **fetch job applications** for a candidate:

```
public with sharing class JobApplicationController {
    @AuraEnabled(cacheable=true) public static List<Job_Application__c>
    getCandidateApplications(Id candidateId) { return [SELECT Id, Job__r.Name,
    Status__c FROM Job_Application__c WHERE Candidate__c = :candidateId]; }
    @AuraEnabled public static Job_Application__c updateApplicationStatus(Id
    appId, String status) { Job_Application__c app = [SELECT Id, Status__c FROM
    Job_Application__c WHERE Id = :appId LIMIT 1]; app.Status__c = status;
    update app; return app; } }
```

Apex Class
JobApplicationController [Help for this Page](#)

Apex Class Detail [Edit](#) [Delete](#) [Download](#) [Security](#) [Show Dependencies](#)

Name	JobApplicationController	Status	Active
Namespace Prefix		Code Coverage	0% (0/10)
Created By	T. Rupasree, 9/21/2025, 12:11 AM	Last Modified By	T. Rupasree, 9/22/2025, 11:26 PM

[Class Body](#) [Class Summary](#) [Version Settings](#) [Trace Flags](#)

```

1 public with sharing class JobApplicationController {
2
3     @AuraEnabled(cacheable=true)
4     public static List<Application__c> getCandidateApplications(Id candidateId) {
5         return [
6             SELECT Id, Job__r.Name, Job__r.Application_Status__c
7             FROM Application__c
8             WHERE Candidate__c = :candidateId
9         ];
10    }
11
12    @AuraEnabled
13    public static Application__c updateApplicationStatus(Id applicationId, String newStatus) {
14        // Fetch the Application along with related Job
15        Application__c app = [
16            SELECT Id, Job__c
17            FROM Application__c
18            WHERE Id = :applicationId
19            LIMIT 1
20        ];
21
22        // Update the status on the related Job
23        Job__c jobToUpdate = [SELECT Id, Application_Status__c FROM Job__c WHERE Id = :app.Job__c LIMIT 1];
24        jobToUpdate.Application_Status__c = newStatus;
25        update jobToUpdate;
26
27        return app;
28    }
29 }

```

[Edit](#) [Delete](#) [Download](#) [Security](#) [Show Dependencies](#)

Notes:

- `@AuraEnabled(cacheable=true)` → for fetching data, allows Lightning Data Service caching.
- `@AuraEnabled` → for DML operations (insert/update).

The screenshot shows a Visual Studio Code editor with a Salesforce project named 'SkillDevEmploymentPortal'. The Explorer pane on the left shows the project structure, including folders for 'flexpages', 'layouts', 'lwc', 'applicationTracker', 'candidateApplications', 'candidateProfileCard', 'helloportal', and 'jobRecommendationList'. The 'jobRecommendationList' folder is selected, showing files like 'jobRecommendationList.html', 'jobRecommendationList.js', and 'jobRecommendationList.js-meta.xml'. The main editor area shows the content of 'jobRecommendationList.js', which imports 'LightningElement' and 'wire' from 'lwc', and defines a 'jobRecommendationList' class that extends 'LightningElement' and uses the 'getRecommendedJobs' method from 'SkillAssignmentHelper'. The bottom panel shows the 'TERMINAL' tab with the following commands and output:

```

PS C:\Users\ADMIN\Desktop\salesforceprojects\SkillDevEmploymentPortal> sfdx force:source:deploy -p force-app/main/default/lwc/jobRecommendationList
jobRecommendationList LightningComponentBundle force-app/main/default/lwc/jobRecommendationList\jobRecommendationList.html
jobRecommendationList LightningComponentBundle force-app/main/default/lwc/jobRecommendationList\jobRecommendationList.js
jobRecommendationList LightningComponentBundle force-app/main/default/lwc/jobRecommendationList\jobRecommendationList.js-meta.xml
Deploy Succeeded.
PS C:\Users\ADMIN\Desktop\salesforceprojects\SkillDevEmploymentPortal>

```

2 LWC: Child Component (Job Card)

```
jobCard.html U x jobCard.js U jobCard.js-meta.xml U CandidateID__c.field-meta.xml 1 ContactNumber...

force-app > main > default > lwc > jobCard > jobCard.html > ...
1 <template>
2   <lightning-card title={job.Job__r.Name}>
3     <p>Status: {job.Status__c}</p>
4     <lightning-button label="Approve" onclick={approve}></lightning-button>
5     <lightning-button label="Reject" onclick={reject}></lightning-button>
6   </lightning-card>
7 </template>
8
```

```
jobCard.html U jobCard.js U jobCard.js-meta.xml U CandidateID__c.field-meta.xml 1 ContactNumber__c.field-meta.xml 1 applicationTracker.js-meta.xml U helloportal.js U

force-app > main > default > lwc > jobCard > jobCard.js > ...
1 import { LightningElement, api } from 'lwc';
2 import updateApplicationStatus from '@salesforce/apex/JobApplicationController.updateApplicationStatus';
3
4 export default class JobCard extends LightningElement {
5   @api job;
6
7   approve() {
8     this.updateStatus('Approved');
9   }
10
11   reject() {
12     this.updateStatus('Rejected');
13   }
14
15   updateStatus(status) {
16     updateApplicationStatus({ appId: this.job.Id, status })
17       .then(result => {
18         // Send event to parent
19         this.dispatchEvent(
20           new CustomEvent('statuschange', {
21             detail: {
22               jobId: result.Id,
23               status: result.Status__c
24             }
25           )
26         );
27       })
28       .catch(error => {
29         console.error('Error updating status', error);
30       });
31   }
32 }
```

```
Terminal Help SkillDevEmploymentPortal

jobCard.html U jobCard.js U jobCard.js-meta.xml U CandidateID__c.field-meta.xml 1 ContactNumber__c.field-meta.xml 1 applicationTracker.js-meta.xml U helloportal.js U

force-app > main > default > lwc > jobCard > jobCard.js-meta.xml > ...
1 <?xml version="1.0" encoding="UTF-8"?>
2 <LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
3   <apiVersion>64.0</apiVersion>
4   <isExposed>false</isExposed>
5   <masterLabel>Job Card</masterLabel>
6   <description>Child component for displaying individual job details</description>
7 </LightningComponentBundle>
```

Notes:

- `@api job` → receives job record from parent.
- `CustomEvent` → lets parent know the status changed.

3 LWC: Parent Component (Job List / Application Tracker)

```

eProfileCard.js U  jobRecommendationList.html U  jobRecommendationList.js U  applicationTracker.html U X
force-app > main > default > lwc > applicationTracker > applicationTracker.html > template > lightning-card
1  <template>
2    <lightning-card title="Application Tracker">
3      <lightning-button label="Submit" onclick={handleSubmit}></lightning-button>
4    </lightning-card>
5  </template>

```

Apex Class

SkillAssignmentHelper

Help for this Page ?

Apex Class Detail Edit Delete Download Security Show Dependencies

Name	Status	Active
SkillAssignmentHelper		
Namespace Prefix	Code Coverage	0% (0/5)
Created By	Last Modified By	
T. Rupasree , 9/21/2025, 12:32 AM	T. Rupasree , 9/22/2025, 11:59 PM	

Class Body Class Summary Version Settings Trace Flags

```

1  public with sharing class SkillAssignmentHelper {
2
3    // Your existing methods...
4
5    @AuraEnabled(cacheable=true)
6    public static List<Job__c> getRecommendedJobs() {
7      try {
8        // Example: fetch top 10 open jobs
9        return [
10         SELECT Id, Name, Location__c, Application_Status__c
11         FROM Job__c
12         WHERE Application_Status__c = 'Open'
13         LIMIT 10
14       ];
15     } catch (Exception e) {
16       System.debug('Error in getRecommendedJobs: ' + e.getMessage());
17       return new List<Job__c>();
18     }
19   }
20 }

```

Edit Delete Download Security Show Dependencies

```

eProfileCard.js U  jobRecommendationList.html U  jobRecommendationList.js U X  applicationTracker.html U  D A
force-app > main > default > lwc > jobRecommendationList > jobRecommendationList.js > ...
1  import { LightningElement, wire } from 'lwc';
2  import getRecommendedJobs from '@salesforce/apex/SkillAssignmentHelper.getRecommendedJobs';
3
4
5  export default class JobRecommendationList extends LightningElement {
6    @wire(getRecommendedJobs) jobs;
7  }
8

```

```

0  CandidateID__c.field-meta.xml 1  ContactNumber__c.field-meta.xml 1  ApplicationTracker.js-me
rce-app > main > default > lwc > applicationTracker > ApplicationTracker.js-meta.xml > ...
1  <?xml version="1.0" encoding="UTF-8"?>
2  <LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
3      <apiVersion>64.0</apiVersion>
4      <isExposed>true</isExposed>
5      <targets>
6          <target>lightning__RecordPage</target>
7          <target>lightning__AppPage</target>
8          <target>lightning__HomePage</target>
9      </targets>
10 </LightningComponentBundle>
11

```

Notes:

- `for:each` with `key={job.Id}` is **mandatory** in LWC iterators.
- `handleStatusChange` updates the UI when a child component dispatches a status change event.

4 Key Concepts in This Apex + LWC Event Flow

Concept	Explanation
Apex Method	Provides server-side data (queries/updates Salesforce records)
@AuraEnabled	Exposes Apex to LWC
@api	Exposes public property to parent component
CustomEvent	Dispatch from child → parent
detail	Carries payload (e.g., { jobId, status })
for:each / key	Required for iterators in template
.then()	Handles promise from Apex call

This setup **supports three LWCs for your project**:

1. **CandidateProfileCard** → show candidate info, can dispatch event when details updated
 2. **ApplicationTracker** → list of applications, updates when status changes
 3. **JobRecommendationList** → similar pattern, dispatch event when a job is selected
-

8. Events in LWC

Events enable communication between components.

Child-to-Parent Communication (Custom Events)

Use Case:

When a user clicks **Approve / Reject** in `JobCard` (child), the parent `ApplicationTracker` needs to know and update the UI.

Implementation in Child (`jobCard.js`)

```
import { LightningElement, api } from 'lwc'; import updateApplicationStatus from '@salesforce/apex/JobApplicationController.updateApplicationStatus'; export default class JobCard extends LightningElement { @api job; // Record passed from parent approve() { this.updateStatus('Approved'); } reject() { this.updateStatus('Rejected'); } updateStatus(status) { updateApplicationStatus({ appId: this.job.Id, status: status }) .then(result => { // Dispatch event to parent const event = new CustomEvent('statuschange', { detail: { jobId: result.Id, status: result.Application_Status__c } }); this.dispatchEvent(event); }) .catch(error => console.error('Error updating status', error)); } }
```

Key Points:

- `@api job` → receives data from parent.
 - `CustomEvent` → sends payload `{ jobId, status }` to parent.
 - `dispatchEvent` → triggers the event.
-

2 Parent Receives Event

Parent HTML (`applicationTracker.html`)

```
<template if:true={jobs}> <template for:each={jobs} for:item="job"> <c-job-card key={job.Id} job={job} onstatuschange={handleStatusChange}> </c-job-card> </template> </template>
```

Parent JS (`applicationTracker.js`)

```
handleStatusChange(event) { const { jobId, status } = event.detail; //
Update the UI locally this.jobs = this.jobs.map(job => { if (job.Id ===
jobId) { return { ...job, Application_Status__c: status }; } return job;
}); }
```

Key Points:

- `onstatuschange` → listens for event from child.
- `event.detail` → contains payload sent by child.
- Update local `@track jobs` array → ensures UI refresh.

Custom (5)

applicationTracker
candidateProfileCard
helloportal
Job Recommendation List
jobCard

⇒ This is parent

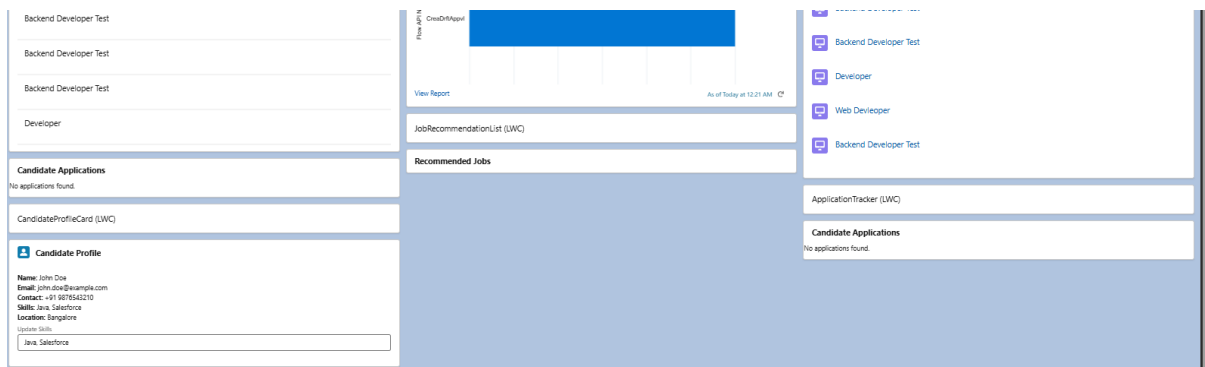
⇒ This is child

3 Optional: Parent-to-Child Communication

- Use `@api` properties in child (`jobCard`) to pass **updated data** from parent if needed.
- Example: `@api job` in child automatically receives updates when parent modifies the array.

4 Testing Events

1. Deploy both **parent and child LWCs**.
2. Add parent component to **Lightning page**.
3. Click **Approve/Reject** in child → check if:
 - Parent UI updates immediately.
 - Record is updated in Salesforce via Apex.



- **Use Case in Portal:**
 - Candidate updates profile → ProfileUpdated event → Refresh Candidate Dashboard.
 - Employer posts new job → NewJobPosted event → Updates Employer Dashboard and sends notification.
 - **Implementation:**
 - **Custom Events** used for LWC-to-LWC communication.
 - **Pub/Sub Events** for communication between unrelated components.
-

9. Wire Adapters

- **Wire adapters** are reactive mechanisms in Lightning Web Components to fetch Salesforce data **without manually calling Apex every time**.
- They are **imported from** `lwc` or `lightning/ui*Api` modules.
- When the data changes in Salesforce, the UI **automatically updates**.

Key points:

- Wire is **reactive**. If a property used in the wire changes, the wire runs again.
 - Used for **calling Apex methods** or **standard Salesforce objects**.
-

1 Wire with Apex Method

Apex Class

```
public with sharing class JobApplicationController {
    @AuraEnabled(cacheable=true) public static List<Application__c>
    getCandidateApplications(Id candidateId) { return [SELECT Id, Job__c,
    Application_Status__c FROM Application__c WHERE Candidate__c =
    :candidateId]; } }
```

LWC JS

```
import { LightningElement, wire, track } from 'lwc'; import
getCandidateApplications from
'@salesforce/apex/JobApplicationController.getCandidateApplications';
export default class ApplicationTracker extends LightningElement { @track
jobs; candidateId = 'YOUR_CANDIDATE_ID_HERE';
@wire(getCandidateApplications, { candidateId: '$candidateId' })
wiredApplications({ error, data }) { if (data) { this.jobs = data; } else
if (error) { console.error(error); } } }
```

Notes:

- Use `$candidateId` to make it **reactive**. If `candidateId` changes, wire re-runs.

- `@wire` can be used **with a function** (for processing) or **with a property** (direct assignment).
-

2 Wire with Salesforce UI API (No Apex Needed)

Lightning provides **built-in wire adapters**:

a. Get Record

```
import { LightningElement, wire } from 'lwc'; import { getRecord } from 'lightning/uiRecordApi'; import NAME_FIELD from '@salesforce/schema/Candidate__c.Candidate_Name__c'; import EMAIL_FIELD from '@salesforce/schema/Candidate__c.Email__c'; export default class CandidateCard extends LightningElement { recordId = 'YOUR_CANDIDATE_ID_HERE'; @wire(getRecord, { recordId: '$recordId', fields: [NAME_FIELD, EMAIL_FIELD] }) candidate; }
```

- `candidate.data` → contains record fields
- `candidate.error` → error details

b. Get Records List

```
import { LightningElement, wire, track } from 'lwc'; import { getListUi } from 'lightning/uiListApi'; import APPLICATION_OBJECT from '@salesforce/schema/Application__c'; export default class ApplicationList extends LightningElement { @track applications; @wire(getListUi, { objectApiName: APPLICATION_OBJECT, listViewApiName: 'All Applications' }) wiredList({ error, data }) { if (data) { this.applications = data.records.records; } else if (error) { console.error(error); } } }
```

- This avoids Apex if you just need **standard queries**.
-

3 Handling Reactive Changes

- If a **parameter changes**, the wire runs again automatically.

```
@track candidateId; @wire(getCandidateApplications, { candidateId: '$candidateId' }) wiredApplications({ error, data }) { ... }
```

- Change `candidateId` dynamically, and **jobs list updates without a refresh**.
-

4 Key Tips for Wire Adapters

1. Use **@track** for reactive properties if you modify them.
2. **Function vs Property**:

- Property: `@wire(getCandidateApplications, { candidateId: '$candidateId' }) jobs;`
- Function: `@wire(getCandidateApplications, { candidateId: '$candidateId' }) wiredJobs({error, data}) { ... }`
- 3. Wire is **read-only**. To modify records, use **Apex imperative calls** or **lightning/uiRecordApi** `updateRecord`.
We Already implemented this in our previous steps

- **Use Case in Portal:**
- Displaying real-time list of available jobs from Job__c object.
- Showing candidate certifications directly from Certification__c object.
- **Implementation:**
- `@wire(getRecord, { recordId: '$recordId', fields: [...] })`
- Used in components like JobDetail and CandidateProfileCard.

10. Imperative Apex Calls

When more control is required, imperative Apex calls are used.

```

Class Body | Class Summary | Version Settings | Trace Flags
1 public with sharing class JobApplicationController {
2
3     @AuraEnabled(cacheable=true)
4     public static List<Application__c> getCandidateApplications(Id candidateId) {
5         return [
6             SELECT Id, Job__c, Candidate__c
7             FROM Application__c
8             WHERE Candidate__c = :candidateId
9         ];
10    }
11
12    @AuraEnabled
13    public static Application__c updateApplicationStatus(Id appld, String status) {
14        // Get the application record
15        Application__c app = [SELECT Id, Job__c FROM Application__c WHERE Id = :appld LIMIT 1];
16
17        // Get the related Job record via lookup
18        Job__c job = [SELECT Id, Application_Status__c FROM Job__c WHERE Id = :app.Job__c LIMIT 1];
19
20        // Update status
21        job.Application_Status__c = status;
22        update job;
23
24        return app;
25    }
26 }

```

Imperative Apex is calling an Apex method **manually from JavaScript**, rather than automatically via the `@wire` service.

- **@wire** → reactive, runs automatically when component loads or parameters change.

- **Imperative Apex** → explicit, you control when the call happens, e.g., on button click, user interaction, or after some calculation.
-

2 When to Use

- You want to call Apex **only when user clicks a button** (not automatically).
- You need to pass **dynamic parameters at runtime**.
- You want to handle **promise results** (success or error) in JS.

Example scenarios:

- Approving or rejecting a job application on button click.
 - Fetching filtered data based on user input.
 - Performing complex operations and updating the UI after completion.
-

3 How It Works in LWC

1. Apex Class:

```
public with sharing class JobApplicationController { @AuraEnabled public
static Application__c updateApplicationStatus(Id appId, String status) {
Application__c app = [SELECT Id, Application_Status__c FROM Application__c
WHERE Id = :appId LIMIT 1]; app.Application_Status__c = status; update app;
return app; } }
```

- `@AuraEnabled` exposes this method to LWC.
- Non-cacheable, because it updates data.

2. LWC JS (Imperative Call):

```
import updateApplicationStatus from
'@salesforce/apex/JobApplicationController.updateApplicationStatus';
updateStatus() { updateApplicationStatus({ appId: this.job.Id, status:
'Approved' }) .then(result => { console.log('Status updated:',
result.Application_Status__c); }) .catch(error => { console.error('Error:',
error); }); }
```

- Call **directly on button click** or any event.
 - Returns a **Promise**, allowing `.then()` for success and `.catch()` for errors.
-

4 Difference from @wire

Feature	@wire	Imperative
Call Timing	Automatic	Manual
Parameters	Fixed or reactive	Dynamic at runtime
Usage	Display data reactively	Actions on user interaction
Return	Reactive properties	Promise

Summary:

Imperative Apex gives you **full control** over when and how Apex methods are called from your LWC. It's essential for **actions like approving/rejecting applications**, where you only want to call Apex when a user clicks a button, not automatically on component load.

- **Use Case in Portal:**
- Candidate clicks "Apply Now" → Imperative call inserts new Application__c record.
- Employer rejects candidate → Imperative call updates Application__c status.
- **Implementation:**
- `applyForJob({ candidateId, jobId })` method invoked via imperative call.
- Success/error handled with toast messages.

11. Navigation Service

Navigation Service enables smooth navigation between Salesforce pages.

The **Lightning Navigation Service** in LWC allows you to navigate to different Salesforce pages **programmatically**, such as:

- Record pages
- Object list views

- Web pages (external URLs)
- Custom tabs / components

It's similar to routing in traditional web apps but adapted for Salesforce Lightning.

1 Key Concepts

- You **import** `NavigationMixin` from `lightning/navigation`.
 - Use `this[NavigationMixin.Navigate](pageReference)` to navigate.
 - `pageReference` is a **JavaScript object** describing the target page.
-

2 Example Usage

a) Import NavigationMixin

```
import { LightningElement } from 'lwc'; import { NavigationMixin } from
'lightning/navigation'; export default class MyComponent extends
NavigationMixin(LightningElement) { navigateToRecord() { // Code here } }
```

b) Navigate to a Record Page

```
this[NavigationMixin.Navigate]({ type: 'standard__recordPage', attributes:
{ recordId: '0015g00000XXXXXX', // replace with actual record Id
objectApiName: 'Account', actionName: 'view' } });
```

Explanation:

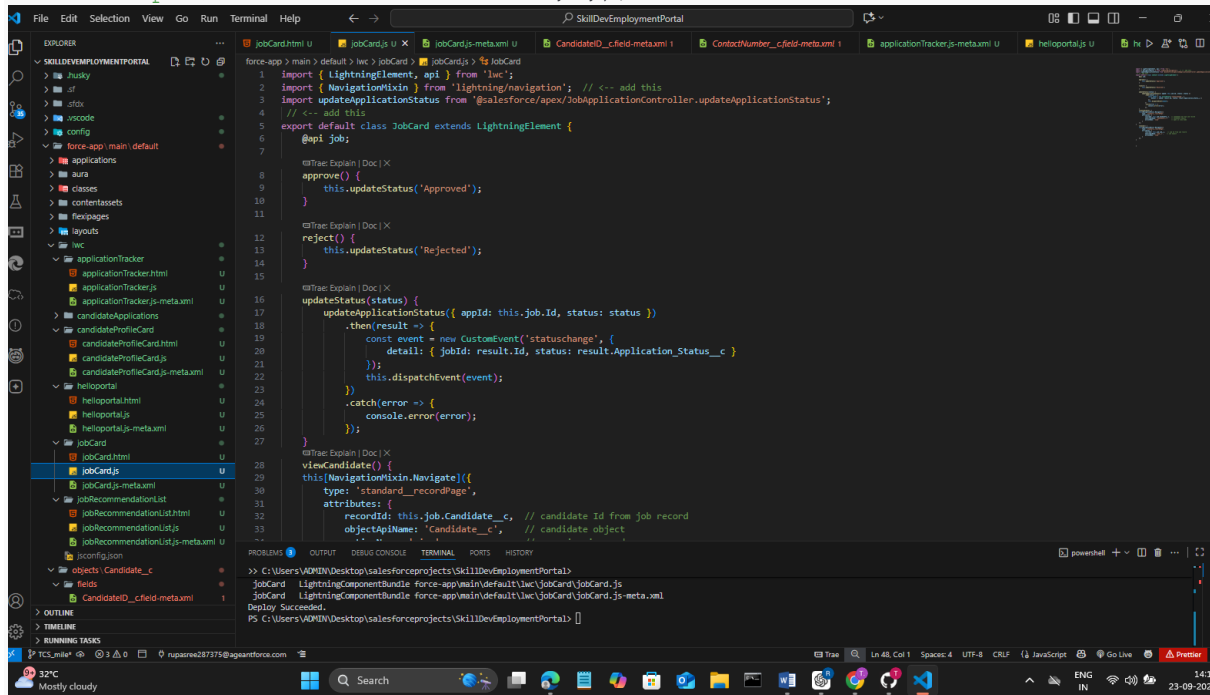
- `type: 'standard__recordPage'` → standard Salesforce record page
 - `recordId` → the record you want to view
 - `objectApiName` → object API name
 - `actionName` → 'view', 'edit', 'clone'
-

c) Navigate to an Object List View

```
this[NavigationMixin.Navigate]({ type: 'standard__objectPage', attributes:
{ objectApiName: 'Application__c', actionName: 'list' }, state: {
filterName: 'All' // optional: filter list view } });
```

d) Navigate to a Web Page (External URL)

```
this[NavigationMixin.Navigate]({ type: 'standard__webPage', attributes: {  
url: 'https://www.salesforce.com' } });
```



3 Integration Example

You could add a button in your `JobCard` LWC:

```
<lightning-button label="View Candidate"  
onclick={viewCandidate}></lightning-button>
```

And JS:

```
viewCandidate() { this[NavigationMixin.Navigate]({ type:  
'standard__recordPage', attributes: { recordId: this.job.Candidate__c,  
objectApiName: 'Candidate__c', actionName: 'view' } }); }
```

- Clicking this button will navigate to the candidate's record page.

4 Notes

- Always extend `NavigationMixin` in your **component class**.
- `NavigationMixin.GenerateUrl` can generate a URL without navigating immediately.

- Works for **standard and custom objects, record pages, web pages, and app pages**.
 - **Use Case in Portal:**
 - Candidate clicks on "View Job" → Redirects to Job Record Page.
 - Employer clicks on "View Candidate" → Redirects to Candidate Record Page.
 - **Implementation:**
 - `NavigationMixin.Navigate` used in LWC to move between record pages, list views, or custom tabs.
-

Conclusion

- ⇒ Phase 6 ensures that the **Skill Development and Employment Portal** is not only functionally strong but also **visually intuitive, responsive, and user-friendly**. Using Salesforce UI capabilities (App Builder, Record Pages, Tabs, Utility Bar) along with **Lightning Web Components integrated with Apex**, we provide a **seamless candidate-employer-admin experience**.
- ⇒ This phase transforms backend data and logic into a **real-world, interactive portal** that enhances user engagement, simplifies workflows, and bridges the gap between **job seekers and employers**.